

Beyond F1: LLM-Based Identifier Renaming and Readability Evaluation for Generated Tests

Jason Liu

Master Student Software Engineering

University of Antwerp

Antwerp, Belgium

jason.liu@student.uantwerpen.be

Abstract—Automated test generation tools such as EvoSuite and Randoop produce unit tests populated with uninformative identifiers such as `var0` and `test42`, which impede comprehension and reduce the practical value of the generated output for the developers who must maintain them. Renaming these identifiers to meaningful names is a necessary post-processing step, yet existing non-LLM approaches achieve at most $F1 = 0.308$ and no scalable readability evaluation framework exists. Token-level accuracy and holistic readability are not the same thing: fine-tuning improves F1 to 0.839 yet simultaneously reduces the LLM readability score, while a zero-shot proprietary model achieves the highest readability of all variants despite a comparable F1, showing that a model can learn to reproduce reference labels without producing names that are more readable to a developer. Neither metric alone is therefore a sufficient ground truth, and both should be reported together when evaluating identifier renaming tools; to support this we contribute CodeReader, a configurable open-source readability grading tool, and demonstrate that even minimal fine-tuning of 15% of training steps captures most of the token-level benefit.

Index Terms—Large Language Models, Automated test generation, Identifier renaming, Test readability, Fine-tuning, LLM-as-a-Judge, Unit tests, Java, Code comprehension

I. INTRODUCTION

Unit tests play a central role in modern software development. They verify that individual units of code behave as intended, serve as a safety net against regressions, and act as executable documentation for engineers studying an unfamiliar system [1], [2]. Beyond correctness, tests must also be *readable*: a test that cannot be understood quickly cannot be trusted, debugged, or extended [3]. Readable tests reduce the cognitive load of fault localization and help developers understand the intended behavior of the system under test without consulting production code [4].

Writing such tests by hand is labor-intensive, which has driven wide adoption of automated test generation tools such as EvoSuite [5] and Randoop [6], [7]. EvoSuite applies genetic algorithms to maximise a coverage criterion such as branch coverage [8]; Randoop uses feedback-directed random testing to generate and filter sequences of method calls [7]. Both tools operate purely on structural and execution properties and have no awareness of semantic meaning, so the tests they produce are populated with generic identifiers such as `var0`, `int0`, and `test1` [9], [10].

This readability deficit has measurable consequences. Butler et al. showed that poor identifier naming degrades code

comprehension and maintainability [11], and Lin et al. found that identifier quality in test code is substantially lower than in production code, with only 61.6% of test identifiers rated acceptable compared with 92.5% in production code [12]. Winkler et al.’s systematic mapping study confirmed that uninformative identifiers are one of the primary factors impeding comprehension of automatically generated tests, and that no single metric adequately captures all dimensions of test readability [13].

Prior work has approached the problem from several angles. DeepTC-Enhancer [3] combined template-based and deep learning methods to improve test readability, though 48% of suggested variable names only *somewhat* conveyed the intended meaning, and the evaluation relied on a small human study that cannot scale to large test suites. De Keersmaecker [14] evaluated GGNN and RefBERT adapted for generated tests and found that neither achieved consistent, concise naming across full test files. Most recently, Biagiola et al. [4] used LLMs to rename identifiers in EvoSuite-generated tests and reported readability on par with developer-written tests in a study with ten professional developers, a promising result, but one that relies on closed proprietary models and does not evaluate fine-tuned open-source alternatives.

A complementary challenge is *evaluation*: measuring whether a renaming actually improves readability. Reference-based metrics such as BLEU [15] are unsuitable because multiple valid names can exist for the same variable, so a correct rename may score near zero against a single reference [16], [17]. The LLM-as-a-Judge paradigm addresses this by using language models as evaluators that assess semantic quality against explicit rubrics, achieving over 80% agreement with human preferences on code-related tasks [18], [19]. Single-judge evaluation is, however, susceptible to model-specific biases such as verbosity preference and position sensitivity [18]; aggregating scores across an ensemble of diverse judge models has been shown to reduce this bias and improve alignment with human judgment [20], [21].

These results suggest that LLMs are a promising vehicle for identifier renaming and for evaluating renamed output at scale, yet key questions remain open. This paper addresses the following research questions:

- **RQ1:** Does an LLM-based renaming pipeline outperform the non-LLM baselines (GGNN and RefBERT)

on identifier naming quality in automatically generated Java tests?

- **RQ2:** How does the amount of fine-tuning affect naming quality across checkpoints (15%, 25%, 50%, 75%, 100%)?
- **RQ3:** Does token-level improvement (F1) correlate with perceived readability as measured by the LLM readability score?

II. BACKGROUND

A. Automated Test Generation

EvoSuite [5] evolves a population of test cases using a genetic algorithm that maximises a coverage criterion such as branch coverage. Randoop [6], [7] generates sequences of method calls incrementally, using execution feedback to discard failing sequences and retain those that extend coverage. Both tools operate purely on structural and execution properties, with no awareness of semantic meaning, so the identifiers they emit are generic and uninformative: local variables receive names such as `double0` or `calculator0`, and test methods receive names such as `test0` or `test42` [9], [10]. Examples are shown in Listing 3 and Listing 4.

B. Identifier Quality and Test Readability

A high-quality identifier satisfies three criteria [14]: *consistency* (no synonyms or homonyms across the codebase), *correctness* (the name corresponds to the concept it represents), and *conciseness* (the name is no more general than the concept warrants). Butler et al. demonstrated empirically that poor identifier naming degrades a developer’s ability to understand and maintain code [11], and Lin et al. found that test code identifiers are substantially lower quality than production code identifiers, with only 61.6% rated acceptable compared to 92.5% for production code [12]. Test method names carry an additional communicative burden: the established JUnit convention encodes the scenario and expected outcome directly in the name using a pattern such as `methodName_stateUnderTest_expectedBehavior` [1]. When generated tests use names like `test42`, neither purpose is served.

Of the three criteria, *consistency* occupies a special position. A name that is individually readable but inconsistent with the surrounding codebase’s naming conventions may be harder to work with than a generic name that follows a predictable pattern, because developers rely on naming regularity to navigate and reason about code quickly [11], [22]. Winkler et al.’s systematic mapping study confirmed that identifier consistency is among the primary factors influencing comprehension of test code, and that no single metric adequately captures all dimensions of test readability [13].

Crucially, however, evaluating consistency requires knowledge of the broader codebase rather than of the test method in isolation. Unlike correctness and conciseness, which can be assessed from the method body alone, consistency can only be judged against the naming conventions established across the surrounding code. This makes it orthogonal to the per-method readability scoring approach used in this work, which evaluates each test method as a self-contained unit. Consistency is

therefore treated as a separate dimension of readability that is left to future work, where a domain-specialized judge with access to codebase history would be required to assess it reliably.

C. Large Language Models for Code

LLMs are transformer-based models pre-trained on large corpora of code and natural language [23]. This pre-training gives them an implicit understanding of naming conventions and the relationship between code structure and semantic intent, which task-specific models had to be explicitly engineered to capture [24], [25]. Code-specialized models push this further: DeepSeek-Coder is trained on 2 trillion tokens of which 87% is source code [26], while Qwen2.5-Coder is pre-trained on over 5.5 trillion tokens spanning 92 programming languages and achieves state-of-the-art performance across more than 10 code benchmarks [27]. Both models have been shown to perform rename refactorings successfully, demonstrating that they have internalized sufficient naming knowledge from their training data [28]. LLM performance on code tasks is significantly affected by the completeness of contextual information available during inference [29], which motivates providing the full function body alongside the identifier list. Few-shot prompting improves output reliability [23], and when a response is malformed the Reflexion retry strategy feeds the failed response back to the model with a concrete error description, substantially increasing the probability of a valid response on the next attempt [30], [31].

D. Fine-Tuning LLMs

Pre-trained LLMs can be adapted to a specific task by continuing training on a small, task-relevant dataset, a process known as fine-tuning [32], [33]. Fine-tuning allows the model to specialize its learned representations for the target domain while retaining the broad knowledge acquired during pre-training. In the context of identifier renaming for generated tests, fine-tuning on human-written test code that has been obfuscated to simulate generated output exposes the model to the specific structural patterns and naming conventions of the target domain, potentially improving both accuracy and consistency of suggestions compared to an untuned baseline. The extent to which fine-tuning improves naming quality, measured both by token-level metrics and by LLM-based readability scoring, is one of the central empirical questions of this work.

E. LLM-as-a-Judge Evaluation

Reference-based metrics such as BLEU are unsuitable for evaluating identifier naming because multiple valid names may exist for the same variable, so a correct rename can score near zero against a single reference [15], [16], [17]. The LLM-as-a-Judge paradigm addresses this by using LLMs as evaluators that assess semantic quality directly against explicit rubrics, achieving over 80% agreement with human preferences [18]. Structured rubrics with fine-grained, independently assessable criteria produce more stable scores than vague numeric scales [34], and aggregating across an ensem-

ble of diverse judge models further reduces individual model bias [20].

III. RELATED WORK

A. Identifier Renaming Without LLMs

Early approaches to automated identifier renaming treated source code as a sequence of tokens and applied statistical language models. Mastropaolo et al. conducted a large-scale empirical study comparing an n-gram model, a T5 model, and a BERT-based model (CugLM) for variable renaming in production code, finding that CugLM achieved 63.5% complete match but that all approaches struggled when multiple valid names existed for the same variable [17]. Context2Name [35] used a sequence autoencoder over local token windows to infer variable names from minified JavaScript, achieving 47.5% exact match, but its dependence on surrounding tokens makes it unsuitable for the fully obfuscated identifiers in generated tests.

Structure-aware approaches address this by operating on graph representations of code rather than raw token sequences. Allamanis et al. applied Gated Graph Neural Networks (GGNNs) to Abstract Syntax Trees augmented with dataflow edges, achieving 32.9% subtoken-level precision on production Java code [24]. De Keersmaeker adapted both GGNN and RefBERT for automatically generated tests [14]: RefBERT outperformed GGNN in token-level accuracy, but neither approach produced consistent or concise names across full test files, and both tended to oversimplify concepts and ignore the test scenario in the suggested name. These results of De Keersmaeker serve as the direct baselines for our work.

B. LLM-Based Test Improvement

DeepTC-Enhancer [3] was the first work to directly target readability of automatically generated tests, combining template-based test summarization with deep learning to suggest variable names. A user study reported that 48% of suggestions fully conveyed the intended meaning and 35% somewhat conveyed it, leaving substantial room for improvement. Daka et al. showed that even rule-based synthesis of test method names from coverage criteria can achieve 53% developer agreement, establishing that method naming is a tractable sub-problem [1].

Biagiola et al. proposed the most closely related prior work: they use LLMs to rename identifiers and test method names in EvoSuite-generated tests while explicitly preserving coverage semantics [4]. Their evaluation across nine industrial and open-source LLMs showed that the readability improvements are semantically stable across repetitions, and a human study with ten professional developers found LLM-improved tests as readable as developer-written ones. Our work differs in several important ways. First, Biagiola et al. evaluate a mix of proprietary and open-source models primarily through external APIs; our pipeline targets locally served open-source models loaded via Hugging Face Transformers, which are stateless and process each function in a single, self-contained prompt. This is a stricter setting that isolates the contribution of prompt design rather than relying on model memory or

session state. Second, because our pipeline communicates through a model-agnostic API layer, it is not tied to any specific model or provider: any model that exposes a compatible endpoint can be substituted, making the approach extensible to future open or closed models. Third, we evaluate fine-tuned open-source models alongside untuned ones, a comparison Biagiola et al. do not include. Fourth, our pipeline operates on identifiers extracted from any well-formed Java test method regardless of which generator produced it, rather than being coupled to EvoSuite’s output format. Finally, we contribute a configurable, reproducible readability grading framework that Biagiola et al. do not provide.

C. Readability Evaluation of Test Code

Scalabrino et al. proposed a comprehensive readability model for source code that combines structural features such as line length and nesting depth with textual features such as identifier quality and comment density [36]. Their findings show that automatically generated tests are doubly penalised: they lack comments entirely, and their identifiers are of poor quality, making structural metrics alone insufficient to capture their readability. Takebayashi et al. studied the role of assertion messages and found that expressive assertions are an important but often overlooked readability factor in test code [37]. Winkler et al.’s systematic mapping study confirmed that no single metric captures all dimensions of test readability, and that human studies remain the gold standard but are not scalable [13].

These limitations motivate the LLM-as-a-Judge evaluation design of our grading tool: it directly assesses semantic quality along multiple independently weighted criteria, avoiding the ceiling imposed by reference-based metrics while remaining fully automated and reproducible.

IV. APPROACH

We will discuss both pipeline and readability grading tool.

A. Pipeline

The renaming pipeline, illustrated in Fig. 1, consists of two stages: identifier extraction and LLM-based renaming. In the extraction stage, each test method is isolated from its enclosing class and all identifiers, both variable names and method names, are collected along with their kind. In the renaming stage, the complete obfuscated method body and the extracted identifier list are passed together to the LLM as a single prompt, so that the model has full syntactic context when suggesting new names [24], [29]. The pipeline processes test methods one by one [14], keeping each call self-contained and stateless.

Since LLM outputs can be malformed or structurally invalid, the pipeline supports a configurable retry mechanism: when a response fails schema validation, the failed output and the specific error reason are fed back to the model in a follow-up prompt [30], [31], [38]. Upon a successful response, each suggested name is mapped back to its original location in the function body and the renamed method is reconstructed.

a) Prompt design:

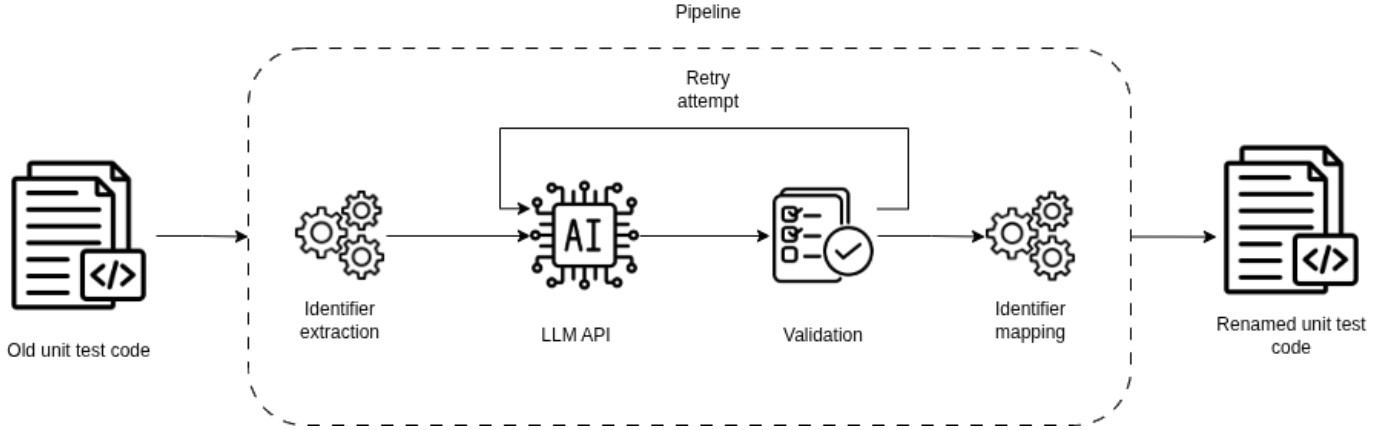


Fig. 1. Renaming pipeline workflow

The pipeline communicates with the LLM through three prompt templates: a system instruction, a user prompt, and a retry prompt. The key design decisions are motivated below.

The user prompt provides the complete obfuscated test method as context alongside the identifier list. This is motivated by the established finding that identifier meaning is inseparable from its surrounding code structure [24]. Prior work has further shown that LLM performance on code tasks is significantly affected by the quality and completeness of contextual information available during inference [29], making full function body provision essential for semantically accurate renaming suggestions.

Each identifier is tagged with its kind: (method) or (variable). Test method names and variable names follow distinct conventions, method names should describe the test scenario in camelCase starting with test [1], while variable names should use concise nouns reflecting the value’s role. The kind tag encodes this distinction directly in the prompt, operationalizing the consistency and conciseness criteria for identifier quality [14].

The prompt constrains the model to respond with a single JSON object mapping original names to new names, including a one-shot example of the expected format [23]. This structured output constraint enables deterministic schema validation, which in turn drives the retry mechanism: when a response fails validation, the pipeline retries with a dedicated prompt that includes the specific error reason and the previous failed response. This design follows the Reflexion framework [30], [31], [38], in which exposing an agent to its prior failure alongside a concrete correction signal substantially increases the probability of a valid response on the next attempt. The maximum number of retries is configurable to account for the non-deterministic nature of LLM outputs. The full prompt templates are provided in Appendix A.1, Appendix A.2, and Appendix A.3.

b) Fine-Tuned Models:

To investigate whether specialization on test code improves renaming quality, we fine-tune a pre-trained LLM on a dataset of obfuscated Java test methods paired with their original identifier names. Fine-tuning is preferred over training from scratch because it allows the model to retain the broad syn-

tactic and semantic knowledge of code acquired during pre-training while specializing its representations for the structural patterns of automatically generated tests [32], [33].

Full fine-tuning of a modern LLM is prohibitively expensive on a single GPU: a 7B parameter model requires roughly 84 GB in 16-bit precision [39]. We therefore apply QLoRA [39], which keeps the base model weights frozen and quantized to 4-bit precision while injecting trainable low-rank adapter matrices into each Transformer layer, reducing trainable parameters by up to four orders of magnitude. More of the training configuration details are provided in Appendix B.

The model is trained as a causal language model on sequences composed of the prompt followed by the completion, where the completion is the JSON mapping of obfuscated names to original names. Prompt tokens are masked in the labels tensor so that the cross-entropy loss is computed exclusively over the predicted identifier names, ensuring the model learns the renaming task rather than reproducing its own input. Samples whose total token length exceeds the maximum context window are discarded rather than truncated, to avoid training on partial completions that would introduce a misleading learning signal.

To reduce overfitting on the fine-tuning dataset, we apply NEFTune [40], which adds uniform random noise to the embedding vectors during each forward pass. NEFTune acts as an implicit regularizer: it has been shown to increase training loss while simultaneously lowering test loss on held-out instructions, directly counteracting memorization of training examples [40]. This property is particularly relevant here, since our fine-tuning dataset consists of obfuscated test methods drawn from a single distribution, making memorization a realistic risk.

The training run saves adapter snapshots at 15%, 25%, 50%, 75%, and 100% of total training steps, yielding five fine-tuned variants alongside the untuned base model. This enables analysis of how naming quality evolves with the amount of fine-tuning, directly addressing RQ2.

B. Readability reading tool

To evaluate the quality of renamed test code, we developed Codereader, an open-source LLM-based readability grading tool available on PyPI [41]. The tool implements the LLM-

as-a-Judge paradigm [18], in which one or more LLMs act as evaluators, assessing the readability of a given test method against a configurable set of domain-specific rules.

The choice of LLM-based evaluation over reference-based metrics such as BLEU is motivated by a fundamental limitation: lexical similarity metrics cannot capture whether an identifier name is semantically appropriate for its context [15], [16], [42], [43]. Since there are often multiple valid names for the same variable, a renamed identifier may score poorly against a reference while still being fully correct in meaning [17]. LLM judges address this by evaluating semantic quality directly, and have been shown to align with human preferences at over 80% agreement when guided by explicit criteria [18].

A critical design decision is that the tool does not ask the judge to produce a free-form score such as “rate this 1 to 100.” Research on LLM-as-a-Judge consistently shows that vague numeric scales produce unstable and poorly calibrated results; structured rubrics with explicit, fine-grained criteria are required for reliable evaluation [18], [34]. Accordingly, codereader is configured through a YAML file that defines a set of named evaluation rules. In our configuration, these rules cover: clarity of variable and method names, behavioral specificity of the test scenario, consistency with JUnit naming conventions, alignment between identifier names and actual test logic, assertion quality, and the presence of unexplained magic numbers. Each rule is expressed as a concrete, actionable criterion rather than a vague dimension, which directly follows the G-Eval recommendation to decompose evaluation into fine-grained, independently assessable steps [34].

To mitigate the variance inherent in single-model evaluation, codereader supports running multiple LLM judges simultaneously with configurable per-model weights. The final score is a weighted aggregation across all active models. The use of an ensemble of judges reduces the impact of individual model bias and improves scoring stability, a strategy supported by the finding that aggregating across multiple evaluators significantly improves alignment with human judgment [20].

The configurations and more information can be found in Listing 1 and Listing 2.

V. EXPERIMENTAL SETUP

A. Dataset

The selected dataset for this study is `methods2test` [44], which is the largest publicly available dataset of its kind, containing approximately 780,000 instances. Each entry in the dataset consists of a mapping between a focal method and its corresponding unit test case, along with metadata such as the class name, package, and source repository. This pairing is a key characteristic of the dataset, as it provides not only the test method itself but also the production code that each test targets.

The dataset defines the operational environment of this study, meaning that the results obtained are assumed to reflect conditions similar to those encountered in practice. A minor reduction of the dataset was necessary prior to use. Upon analysis, several entries were found to be incomplete or malformed. Examples include unclosed braces, unclosed

string literals, and test cases that were truncated mid-function, rendering them entirely unusable.

The test cases were manually obfuscated such that each function name was renamed to `func_n` and each variable name to `var_n`, as illustrated in Listing 5. Each obfuscated test case was then paired with its original version and stored as a JSONL object, as shown in Listing 6.

The evaluation subset consists of 100 randomly sampled test methods drawn with a fixed seed from the held-out test split of `methods2test` [44]. This sample size was determined by a prospective power analysis using the Wilcoxon signed-rank effect-size r [45], [46], computed from a pilot run across all six fine-tuning checkpoints. For every primary comparison, obfuscated vs. renamed and obfuscated vs. oracle, the analysis yields large, highly significant effects ($|r| \geq 0.50$, $p < 0.001$) for which the recommended n never exceeds 10, well within the 100-sample budget.

The one exception is the renamed vs. oracle comparison, which involves a small and fragile effect at several checkpoints (e.g. $r = -0.23$, $p = 0.048$ at 0%, with the weighted-average variant not significant at $p = 0.20$). The recommended n for this comparison reaches up to 2 085 samples at the most demanding checkpoint. Scaling to this budget is infeasible within the hardware and time constraints of this work: at the observed throughput of 3 to 6 hours per 100 samples, a single condition alone would require an estimated 133 hours. Importantly, this instability is itself informative, it reflects that the pipeline produces variable-quality output that is sometimes near-oracle and sometimes substantially below it, a characteristic that additional samples would describe more precisely but not resolve. Full statistical details are provided in Appendix F.

B. Models evaluated

Two models are evaluated in this thesis: Qwen2.5-Coder 7B as a fine-tuned open-source model, and GPT-5 mini as a zero-shot closed-source reference baseline.

Qwen2.5-Coder 7B [27] is a code-specialized open-source model pre-trained on 5.5 trillion tokens across 92 programming languages. At the time of model selection (November 2025), it was the best-performing open-source code model at the 7B parameter scale, achieving state-of-the-art results across standard code benchmarks including HumanEval (88.4), MBPP (83.5), and EvalPlus (81.9), as reported in the original release [27] and detailed further in Appendix J. The 7B variant was chosen over larger variants in the same family based on the hardware available at the time of model selection; fine-tuning was subsequently carried out on a cloud environment, where even the 7B model required approximately 20 GB of VRAM under QLoRA. Its strong benchmark standing, code specialization, and permissive licence made it the most practical base for fine-tuning on our obfuscated test-code dataset using QLoRA [39]. Adapter snapshots are saved at 15%, 25%, 50%, 75%, and 100% of training steps, yielding five fine-tuned variants alongside the untuned base model.

GPT-5 mini [47] is a cost-efficient model from OpenAI’s GPT-5 family, featuring a 400K-token context window, native structured-output support, and built-in reasoning capabilities.

Alongside it, **GPT-4.1 mini** [48] is a non-reasoning model from the GPT-4.1 family targeting similar low-latency, high-throughput workloads. Both are evaluated zero-shot as proprietary baselines. The choice of mini variants is deliberate and mirrors the rationale applied to the open-source side: rather than benchmarking only frontier-scale systems, this thesis focuses on lightweight, cost-efficient models that are practical under real-world resource constraints, both in terms of hardware and API cost.

All evaluated variants are summarized in Table I.

TABLE I
SUMMARY OF EVALUATED MODEL VARIANTS

Model	Params	Fine-tuned	Checkpoint
Qwen2.5-Coder (zero-shot)	7B	No	n/a
Qwen2.5-Coder (15%)	7B	Yes	15%
Qwen2.5-Coder (25%)	7B	Yes	25%
Qwen2.5-Coder (50%)	7B	Yes	50%
Qwen2.5-Coder (75%)	7B	Yes	75%
Qwen2.5-Coder (100%)	7B	Yes	100%
GPT-4.1 mini (zero-shot)	n/a	No	n/a
GPT-5 mini (zero-shot)	n/a	No	n/a

Full details on the compute environment and hardware used are provided in Appendix H.

C. Baselines

We compare against the GGNN [24] and RefBERT [49] techniques as adapted for generated test code by De Keersmaecker [14]. Both are re-evaluated on our subset to ensure a fair comparison under identical data conditions; results are reported in Section VI.

D. Metrics

a) Token-level Metrics:

To enable comparison with the GGNN and RefBERT baselines, we adopt the sub-token-level evaluation protocol established by Allamanis et al. [24] and widely used in code identifier naming research [49], [50]. Since identifiers are composed of sub-tokens (e.g., `getUserList` splits into `{get, user, list}`), precision measures how many predicted sub-tokens are correct, recall measures how many expected sub-tokens were produced, and F1 is their harmonic mean, serving as the primary quantitative metric for cross-model comparison.

Beyond sub-token overlap, exact match measures whether all sub-tokens of a predicted identifier are fully correct. We report both the ordered variant, where sub-token order must match, and the unordered variant, which treats sub-tokens as a set. The latter is motivated by the bag-of-tokens observation of Liu et al. [49]: sub-tokens in an identifier can often be rearranged without changing its meaning (e.g., `defaultVersion` vs. `versionDefault`).

Finally, Character Error Rate (CER) captures the relative number of character-level edit operations needed to transform

a prediction into the reference, providing a fine-grained measure of how far off incorrect predictions are [24].

b) LLM Readability Score:

Token-level metrics such as F1 and exact match evaluate each identifier in isolation against a fixed reference label. This is insufficient for our setting: multiple valid names can describe the same concept, and the collective readability of all renamed identifiers in a test method matters more than individual sub-token overlap [36]. A test method where every identifier achieves high F1 but names are semantically inconsistent is less useful to a developer than one where names cohesively describe the test scenario.

To address this, we adopt an LLM-as-judge approach [18], which has been shown to correlate with human preference at over 80% agreement — on par with the level of agreement between human experts. Applied to code evaluation specifically, ICE-Score [19] demonstrated that instructing LLMs to score generated code achieves superior correlation with human judgement compared to token-matching metrics such as BLEU. Recent work has further shown that LLMs can serve as scalable and human-aligned readability evaluators for both functional code and unit tests specifically [51], motivating our use of an LLM judge to score renamed test methods.

The judge scores each test method on a scale of 0 to 100 based on four weighted criteria derived from established identifier quality research [12], [36]: identifier naming clarity and consistency (weight 0.75), behavioral specificity of the test method name (0.15), and assertion quality (0.10). Test independence is excluded from scoring as it is orthogonal to the identifier renaming task. The full rubric penalizes generic auto-generated names (e.g., `var0`, `obj1`), rewards coherent cross-identifier naming strategies, and requires that identifier names match the actual semantics of what they hold or compute [4].

The judge ensemble consists of three locally-run open-source models. While larger models correlate more closely with human judgement [18], Verga et al. [20] demonstrated that a panel of smaller models from diverse model families outperforms a single large judge and exhibits less intra-model bias, while being substantially more cost-efficient. Our ensemble of three models from disjoint families follows this design principle, and is further supported by evidence that diverse judge ensembles reduce individual model bias in code readability assessment specifically [51].

Several criteria guided the selection of models for the readability grading tool. It is a reading comprehension and evaluation task rather than a code generation task, meaning that strong general language understanding and instruction-following ability are more important than code generation benchmarks such as HumanEval. The secondary requirement was local deployability: all models had to run on the available hardware without exceeding its memory capacity (see Appendix H for the hardware specification). See Appendix K for full details.

The score is averaged across all test methods in the benchmark (`llm_score_avg`). This metric captures what token-level metrics cannot: whether the renamed test as a whole is under-

standable to a developer, making it the primary qualitative metric in our evaluation.

VI. RESULTS & DISCUSSION

In this section we will discuss the three research questions.

A. RQ1: LLM Pipeline vs. Non-LLM Baselines

Table II reports the token-level metrics for both non-LLM baselines, all Qwen2.5-Coder checkpoints, and GPT-5 mini evaluated on the same subset. T1 (GGNN) performs poorly on our subset, achieving an F1 of 0.019 and a CER of 143.9%, indicating that the model almost entirely fails to produce correct subtokens and generates predictions that are substantially longer than the reference names. This result is consistent with De Keersmaecker’s observation that GGNN struggles with obfuscated test code due to the absence of the method under test in the graph representation [14]. T2 (RefBERT) performs considerably better at F1 = 0.308, though its CER of 92.6% still indicates substantial character-level divergence from the reference names.

Looking at LLM-based models, they far surpass the non-LLM baselines. The untuned Qwen2.5-Coder baseline (0%) achieves F1 = 0.760, precision = 0.714, and recall = 0.817, with a CER of 32.0%. GPT-5 mini achieves a comparable F1 of 0.756 with a higher recall of 0.831 but lower precision of 0.699. If we then look at GPT-4.1 mini, it achieves an even better benchmarking despite being a non-reasoning model. We can clearly see a difference in token usage for non-reasoning models compared to reasoning models. The GPT-5 mini model produces on average 562 completion tokens per file compared to 43 to 48 for all Qwen checkpoints and even worse compared to the GPT-4.1 mini results of 39 average completions tokens. Qwen checkpoints improve further, reaching F1 scores between 0.834 and 0.839. These results confirm that the LLM-based pipeline provides a large and consistent improvement over both non-LLM baselines, directly answering RQ1 affirmatively.

TABLE II
TOKEN-LEVEL METRICS FOR BASELINES, QWEN2.5-CODER CHECKPOINTS AND GPT FAMILY ON THE SAME SUBSET.

Model	Precision	Recall	F1	CER (%)	LLM score
T1: GGNN	0.026	0.015	0.019	143.9	63.245
T2: RefBERT	0.320	0.306	0.308	92.6	55.366
Qwen (zero-shot)	0.714	0.817	0.760	32.0	69.96
Qwen (15%)	0.848	0.825	0.834	17.5	65.12
Qwen (25%)	0.853	0.831	0.839	16.9	64.76
Qwen (50%)	0.849	0.828	0.836	17.5	65.825
Qwen (75%)	0.853	0.824	0.836	17.2	63.6
Qwen (100%)	0.849	0.826	0.835	17.3	65.6
GPT-4.1 mini (zero-shot)	0.73	0.835	0.776	31.13	69.38
GPT-5 mini (zero-shot)	0.699	0.831	0.756	35.4	70.233

B. RQ2: Effect of Fine-Tuning Amount

The results in Table III reveal a clear and immediate effect of fine-tuning on token-level metrics: F1 increases sharply from 0.760 at the untuned baseline (0%) to 0.834 at the first checkpoint (15%), after which it plateaus and remains stable across all subsequent checkpoints (25%: 0.839, 50%: 0.836, 75%: 0.836, 100%: 0.835). This suggests that the majority of the fine-tuning benefit is captured very early in training, and that additional training steps beyond 15% yield diminishing returns on token-level naming accuracy.

Regarding the reliability of the effect size estimates, the renamed vs. oracle comparison in Table V shows considerable variance across checkpoints. At 0%, the weighted LLM score comparison yields $r = -0.13$ ($p = 0.20$), which is not statistically significant, while at 50% the same comparison yields $r = -0.22$ ($p = 0.027$), only marginally so. This variance indicates that the gap between renamed and oracle quality is not stable across test files: some files receive near-oracle scores after renaming while others remain substantially below, and this heterogeneity is not resolved by fine-tuning. As discussed in Appendix F, this instability is itself an informative finding rather than a statistical artefact, reflecting the inherent difficulty of producing uniformly high-quality names across structurally diverse test methods.

TABLE III
QWEN2.5-CODER RESULTS PER CHECKPOINT ON THE BENCHMARK SUBSET.

Checkpoint	Precision	Recall	F1	LLM Score
0%	0.714	0.817	0.760	69.96
15%	0.848	0.825	0.834	65.11
25%	0.853	0.831	0.839	64.76
50%	0.849	0.828	0.836	65.83
75%	0.853	0.824	0.836	63.60
100%	0.849	0.826	0.835	65.60

A secondary finding concerns schema compliance. The untuned baseline produced 7 `ExtraKeyError` responses across 100 files, in which the model returned additional identifiers beyond those listed in the prompt. All fine-tuned checkpoints from 15% onward produced zero schema errors, indicating that fine-tuning on the structured JSON output format substantially improves instruction following and eliminates retry overhead in practice.

C. RQ3: F1 vs LLM Readability Scoring

Looking back at Table II, the LLM readability score tells a different story. The untuned baseline (zero-shot) achieves the highest readability score of all variants (avg: 69.96), which then drops at 15% (65.11) and remains consistently lower across all fine-tuned checkpoints. This is a counterintuitive but informative finding: fine-tuning improves how closely the model reproduces the reference labels, yet the judge ensemble rates the output as less readable.

One of the reasons is that fine-tuning causes the model to rename tests closer to how the original codebase named

them. An example of this is a test method that was renamed `testShouldHaveNoFeaturesAfterRemoval` for zero-shot and `shouldRemoveAllFeatures` for 15% fine-tuning. Comparing these to the oracle `removeAllFeatures`, we can clearly see the names converging, and taking this one step further, we see `removeallfeatures` at 25% fine-tuning.

Examining the readability scores reveals a clear degradation trend as fine-tuning increases. The Qwen zero-shot model achieves the highest readability score of 81.0, demonstrating strong out-of-the-box performance on identifier naming. However, as fine-tuning is applied, scores drop substantially: the 15% fine-tuned variant scores 54.0, and the 25% fine-tuned variant scores 50.0. This is further corroborated by the generated names themselves, as the zero-shot model produces more descriptive, intent-revealing names that better communicate what the test is verifying. Comparing this against the GPT family, GPT-4.1 renames the method to `testRemoveAllFeaturesResultsInNoFeatures`, achieving a readability score of 78.0, while GPT-5 produces `testShouldHaveNoFeaturesAfterRemovingAllFeatures` with a score of 75.0. Both GPT models generate grammatically natural, semantically meaningful names that closely follow the given-when-then naming convention.

From this, we can conclude that fine-tuning causes the model to reproduce dataset-specific naming patterns that achieve closer reference token overlap, but are not necessarily more readable by the criteria defined in the CodeReader rubric, such as behavioral specificity and semantic alignment with test logic [36]. The untuned model, unconstrained by those patterns, produces more varied names that the judge rewards more highly, even at the cost of lower F1.

This tension between F1 and the LLM readability score points to a fundamental limitation of reference-based evaluation for identifier naming: a model can score well on F1 by learning dataset-specific naming conventions without producing names that are inherently more readable, a concern previously raised by Mastropaolo et al. [17] and Evtikhiev et al. [16].

D. Discussion

The three research questions together reveal a nuanced picture of LLM-based identifier renaming. RQ1 establishes that even an untuned LLM substantially outperforms dedicated non-LLM approaches, suggesting that the broad code understanding acquired during pre-training is sufficient to transfer meaningfully to the renaming task without task-specific adaptation. RQ2 shows that fine-tuning provides an immediate and significant token-level gain but saturates rapidly, with 15% of training steps capturing most of the benefit, a practically important finding given the computational cost of fine-tuning. The early saturation also suggests that the model quickly learns the structural patterns of the obfuscated dataset, after which additional exposure yields diminishing returns.

The most consequential finding, however, emerges from RQ3: fine-tuning improves F1 while simultaneously reducing the LLM readability score. This divergence suggests that

token-level accuracy and holistic readability are measuring fundamentally different properties. A model that converges towards dataset naming conventions scores higher on F1 but may produce names that are technically closer to the reference while being less descriptive or behaviorally specific than the untuned model’s output. This has direct implications for how future work should evaluate identifier renaming tools: neither F1 alone nor an LLM readability score alone is a sufficient ground truth, and the two metrics should be reported together.

Finally, the oracle comparison shows that the untuned model already produces output that the judge ensemble rates comparably to human-written tests on average, which is a stronger result than token-level metrics alone would suggest.

VII. THREATS TO VALIDITY

This section discusses the potential threats to the validity of the study following the categorization of Wohlin et al. [52], considering construct, internal, external, and conclusion validity.

A. Construct Validity

The primary construct validity threat concerns the LLM readability score. While the scoring criteria are grounded in established identifier quality research [12], [36], the score is produced by an ensemble of smaller open-source judge models rather than human evaluators, and may not fully capture what developers consider readable in practice. The absence of a human study means we cannot verify that judge scores correlate with actual developer preference on our specific dataset.

Three interrelated limitations of the judge ensemble compound this threat. First, the judge models are generative and therefore non-deterministic: scoring the same test method twice under identical conditions may produce different scores. The reported LLM readability scores consequently reflect a single evaluation run per file and per checkpoint, and inter-run variance was not formally measured. The use of an ensemble of three diverse judge models partially mitigates this by producing a more stable aggregated score than any single model [20], but residual non-determinism remains inherent to any LLM-based evaluation approach. Second, the choice of judge models may itself introduce bias: when the ensemble scores the untuned model higher than fine-tuned checkpoints, it is unclear whether this reflects a genuine readability signal or an implicit preference of the judge models for the longer, more descriptive names that the untuned model tends to produce. Third, different judge model selections may produce different relative rankings. Without a human study to validate CodeReader scores against developer preference, these biases cannot be fully ruled out as confounding factors.

B. Internal Validity

Using human-written tests rather than using genuinely auto-generated tests. While this simulates the surface-level naming patterns produced by tools like EvoSuite [5] and Randoop [6], the structural characteristics of truly generated tests differ in ways that matter for this task. Auto-generated tests tend to

be shorter, contain fewer variables, exercise a narrower range of method call patterns, and lack the semantic coherence of human-written code [9]. A renaming model trained and evaluated on obfuscated human-written tests may therefore encounter structurally simpler inputs at deployment time, potentially inflating the F1 and readability scores reported here relative to what would be observed on genuine generator output. Evaluating the pipeline on real EvoSuite and Randoop output is the most direct way to assess this threat and is identified as a priority for future work.

Additionally, the methods2test dataset [44] contains known identifier quality issues [12]: a non-trivial fraction of human-written test identifiers are themselves of poor quality, meaning the oracle labels used for F1 computation are not a reliable upper bound on naming quality. This reinforces the finding from RQ3 that F1 alone is insufficient as an evaluation criterion, since a model converging toward imperfect reference labels may score well on F1 while producing names that are no more readable than the obfuscated originals.

C. External Validity

The evaluation is limited to Java test methods sampled from methods2test [44]; as discussed in Appendix F, this subset is statistically adequate for the primary comparisons but underpowered for the renamed vs. oracle gap at some checkpoints. Beyond sample size, results may not generalize to other programming languages, other test generators, or test methods from different domains. The fine-tuning dataset is drawn from the same distribution as the evaluation subset, so performance on out-of-distribution test structures such as genuinely auto-generated tests from EvoSuite or Randoop remains unknown.

Furthermore, all fine-tuning experiments are conducted with a 7B parameter model due to hardware constraints, and the quantitative results may not generalize to larger open-source models. However, the two central findings of this study are expected to hold more broadly: the benefit of fine-tuning on token-level metrics is well-established across model scales [39], [53], and the divergence between F1 and holistic readability is a property of the metrics themselves rather than of any specific model, as previously noted by Mastropaolo et al. [17] and Evtikhiev et al. [16].

D. Conclusion Validity

As demonstrated in Appendix F, 100 samples is statistically adequate for all primary comparisons that constitute the core empirical contribution of this work. The one exception is the renamed vs. oracle comparison at some checkpoints, where the effect is small and fragile rather than systematically negative, and conclusions about the absolute quality gap between renamed and human-written output should be treated as indicative rather than definitive.

VIII. CONCLUSION

This paper presented an LLM-based pipeline for renaming identifiers in automatically generated Java tests, evaluated against GGNN and RefBERT baselines, six fine-tuning check-

points of Qwen2.5-Coder 7B, GPT-5 mini and GPT-4.1 mini as zero-shot closed-source baselines, all on the same 100-sample subset of the methods2test dataset. All LLM-based baselines outperform the non-LLM baselines on all token-level metrics, with the untuned Qwen baseline achieving $F1 = 0.760$, GPT-4.1 mini achieving $F1 = 0.776$ and GPT-5 mini achieving $F1 = 0.756$, against $F1 = 0.019$ and $F1 = 0.308$ for GGNN and RefBERT respectively. Fine-tuning provides an immediate token-level gain that saturates at 15% of training steps, suggesting that only minimal fine-tuning is needed to achieve most of the benefit. A secondary finding is that fine-tuning eliminates schema compliance errors entirely from 15% onward, improving pipeline reliability in practice.

The most consequential finding is the divergence between token-level F1 and the LLM readability score. Fine-tuning improves F1 while reducing the readability score and GPT-5 mini achieves the highest readability score of all variants despite its F1 being comparable to the untuned Qwen baseline. Together these results suggest that dataset-specific naming conventions and holistic readability are not equivalent properties, and that a model can score well on F1 by learning to reproduce reference labels without producing names that are more readable to a developer. Neither metric alone is a sufficient ground truth, and both should be reported together when evaluating identifier renaming tools. We also contributed CodeReader, a configurable open-source readability grading tool that enables reproducible LLM-based evaluation without requiring a human study.

Future work should evaluate the pipeline on genuinely auto-generated tests from EvoSuite and Randoop, extend the approach to other programming languages, and conduct a human study to validate CodeReader scores against developer preference. During the course of this work, CodeReader was extended to support locally served adapter-based models via Hugging Face Transformers, enabling the fine-tuned Qwen2.5-Coder checkpoints to act as judge models directly. Due to time constraints, this configuration was not evaluated in the current study and remains an avenue for future work: using a domain-specialized judge that was itself fine-tuned on test code naming criteria may improve scoring alignment with developer preference compared to the general-purpose ensemble used here, and may also help resolve the divergence observed between F1 and readability scores.

REFERENCES

- [1] E. Daka, J. M. Rojas, and G. Fraser, "Generating Unit Tests with Descriptive Names or: Would You Name Your Children Thing1 and Thing2?," in *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, New York, NY, USA: ACM, 2017, pp. 57–67. doi: 10.1145/3092703.3092727.
- [2] S. Demeyer, S. Ducasse, and O. Nierstrasz, *Object-Oriented Reengineering Patterns*. Square Bracket Associates, 2009.
- [3] D. Roy et al., "DeepTC-Enhancer: Improving the Readability of Automatically Generated Tests," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, New York, NY, USA: ACM, 2020. doi: 10.1145/3324884.3416622.
- [4] M. Biagiola, G. Ghislotti, and P. Tonella, "Improving the Readability of Automatically Generated Tests using Large Language Models," in *Proceedings of the 2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*, Naples, Italy: IEEE, 2025.

- [5] G. Fraser and A. Arcuri, "EvoSuite: Automatic Test Suite Generation for Object-Oriented Software," in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, New York, NY, USA: ACM, 2011, pp. 416–419. doi: 10.1145/2025113.2025179.
- [6] C. Pacheco and M. D. Ernst, "Randoop: Automatic Unit Test Generation for Java." 2007.
- [7] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-Directed Random Test Generation," in *Proceedings of the 29th International Conference on Software Engineering (ICSE)*, Minneapolis, MN, USA: IEEE Computer Society, 2007, pp. 75–84.
- [8] P. McMinn, "Search-Based Software Testing: Past, Present and Future," in *Proceedings of the 4th International Workshop on Search-Based Software Testing (SBST)*, Berlin, Germany: IEEE, 2011, pp. 153–163. doi: 10.1109/ICSTW.2011.100.
- [9] G. Fraser and A. Arcuri, "EvoSuite: On the Challenges of Test Case Generation in the Real World," in *Proceedings of the 6th IEEE International Conference on Software Testing, Verification and Validation (ICST)*, IEEE, 2013, pp. 362–369. doi: 10.1109/ICST.2013.5954405.
- [10] J. Wang, B. Suleiman, and M. J. Alibasa, "Automated Unit Test Case Generation: A Systematic Literature Review." [Online]. Available: <https://arxiv.org/abs/2504.20357>
- [11] S. Butler, M. Wermelinger, Y. Yu, and H. Sharp, "Exploring the Influence of Identifier Names on Code Quality: An Empirical Study," in *Proceedings of the 14th European Conference on Software Maintenance and Reengineering (CSMR)*, Madrid, Spain: IEEE, 2010, pp. 156–165. doi: 10.1109/CSMR.2010.27.
- [12] B. Lin, C. Nagy, G. Bavota, A. Marcus, and M. Lanza, "On the Quality of Identifiers in Test Code," in *2019 19th International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2019, pp. 204–215. doi: 10.1109/SCAM.2019.00031.
- [13] D. Winkler, P. Urbanke, and R. Ramler, "Investigating the Readability of Test Code: Combining Scientific and Practical Views," *Empirical Software Engineering*, vol. 29, no. 2, p. 53, 2024, doi: 10.1007/s10664-023-10390-z.
- [14] A. De Keersmaecker, "Enhancing Test Code Understandability with Machine Learning-Based Identifier Naming," Master's Thesis, 2024.
- [15] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, Philadelphia, PA, USA: Association for Computational Linguistics, 2002, pp. 311–318. [Online]. Available: <https://aclanthology.org/P02-1040/>
- [16] M. Evtikhiev, E. Bogomolov, Y. Sokolov, and T. Bryksin, "Out of the BLEU: How Should We Assess Quality of the Code Generation Models?," *Journal of Systems and Software*, vol. 203, p. 111741, 2023, doi: 10.1016/j.jss.2023.111741.
- [17] A. Mastropaolo, E. Aghajani, L. Pascarella, and G. Bavota, "Automated Variable Renaming: Are We There Yet?," *Empirical Software Engineering*, vol. 28, no. 2, p. 45, 2023, doi: 10.1007/s10664-022-10274-8.
- [18] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [19] T. Y. Zhuo, "ICE-Score: Instructing Large Language Models to Evaluate Code," in *Findings of the Association for Computational Linguistics: EACL 2024*, Association for Computational Linguistics, 2024, pp. 2232–2242. [Online]. Available: <https://aclanthology.org/2024.findings-eacl.148/>
- [20] P. Verga *et al.*, "Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models." [Online]. Available: <https://arxiv.org/abs/2404.18796>
- [21] C.-M. Chan *et al.*, "ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [22] B. Lin, S. Scalabrino, A. Mocchi, R. Oliveto, G. Bavota, and M. Lanza, "Investigating the Use of Code Analysis and NLP to Promote a Consistent Usage of Identifiers," in *Proceedings of the 17th IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM 2017)*, IEEE, 2017, pp. 81–90. doi: 10.1109/SCAM.2017.13.
- [23] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.
- [24] M. Allamanis, M. Brockschmidt, and M. Khademi, "Learning to Represent Programs with Graphs," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1711.00740>
- [25] A. Fan *et al.*, "Large Language Models for Software Engineering: Survey and Open Problems," in *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, IEEE, 2023, pp. 31–53. doi: 10.1109/ICSE-FoSE59343.2023.00008.
- [26] D. Guo *et al.*, "DeepSeek-Coder: When the Large Language Model Meets Programming — The Rise of Code Intelligence." [Online]. Available: <https://arxiv.org/abs/2401.14196>
- [27] B. Hui *et al.*, "Qwen2.5-Coder Technical Report." [Online]. Available: <https://arxiv.org/abs/2409.12186>
- [28] J. Cordeiro, E. Noei, and Y. Zou, "An Empirical Study on the Code Refactoring Capability of Large Language Models," *ACM Transactions on Software Engineering and Methodology*, 2024, doi: 10.1145/3801158.
- [29] Z. Wang, L. Zhang, C. Cao, N. Luo, X. Luo, and P. Liu, "How Does Naming Affect Language Models on Code Analysis Tasks?," *Journal of Software Engineering and Applications*, vol. 17, no. 11, pp. 803–816, 2024, doi: 10.4236/jsea.2024.1711044.
- [30] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [31] R. Ravi, D. Bradshaw, S. Ruberto, G. Jahangirova, and V. Terragni, "LLMLOOP: Improving LLM-Generated Code and Tests through Automated Iterative Feedback Loops," in *Proceedings of the 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, 2025.
- [32] A. Hosna, E. Merry, J. Gyalmo, Z. Alom, Z. Aung, and M. A. Azim, "Transfer Learning: A Friendly Introduction," *Journal of Big Data*, vol. 9, no. 1, p. 102, 2022, doi: 10.1186/s40537-022-00652-w.
- [33] K. W. Church, Z. Chen, and Y. Ma, "Emerging Trends: A Gentle Introduction to Fine-Tuning," *Natural Language Engineering*, vol. 27, no. 6, pp. 763–778, 2021, doi: 10.1017/S1351324921000322.
- [34] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Singapore: Association for Computational Linguistics, 2023, pp. 2511–2522. doi: 10.18653/v1/2023.emnlp-main.153.
- [35] R. Bavishi, M. Pradel, and K. Sen, "Context2Name: A Deep Learning-Based Approach to Infer Natural Variable Names from Usage Contexts." [Online]. Available: <https://arxiv.org/abs/1809.05193>
- [36] S. Scalabrino, M. Linares-Vásquez, R. Oliveto, and D. Poshyvanyk, "A Comprehensive Model for Code Readability," *Journal of Software: Evolution and Process*, vol. 30, no. 6, p. e1958, 2018, doi: 10.1002/smr.1958.
- [37] T. Takebayashi, A. Peruma, M. W. Mkaouer, and C. D. Newman, "An Exploratory Study on the Usage and Readability of Messages within Assertion Methods of Test Cases." 2023.
- [38] J. Jiang, J. Shen, S. Kim, K. M. Yoo, J. Kim, and S. Kim, "ReflexiCoder: Teaching Large Language Models to Self-Reflect on Generated Code and Self-Correct It via Reinforcement Learning." [Online]. Available: <https://arxiv.org/abs/2603.05863>
- [39] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [40] N. Jain *et al.*, "NEFTune: Noisy Embeddings Improve Instruction Finetuning," in *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024. [Online]. Available: <https://openreview.net/forum?id=0bMmZ3fkCk>
- [41] J. Liu, "codereader: A Tool for Grading Unit Test Readability Using LLMs." 2025.
- [42] C. Callison-Burch, M. Osborne, and P. Koehn, "Re-evaluating the Role of BLEU in Machine Translation Research," in *Proceedings of the 11th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, Trento, Italy: Association for Computational Linguistics, 2006, pp. 249–256. [Online]. Available: <https://aclanthology.org/E06-1032/>

- [43] E. Reiter, "A Structured Review of the Validity of BLEU," *Computational Linguistics*, vol. 44, no. 3, pp. 393–401, 2018, doi: 10.1162/coli_a_00322.
- [44] M. Tufano, S. K. Deng, N. Sundaresan, and A. Svyatkovskiy, "Methods2Test: A Dataset of Focal Methods Mapped to Test Cases," in *Proceedings of the 19th International Conference on Mining Software Repositories*, in MSR '22. ACM, May 2022.
- [45] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Hillsdale, NJ: Lawrence Erlbaum Associates, 1988.
- [46] A. Arcuri and L. Briand, "A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering," in *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, ACM, 2011, pp. 1–10. doi: 10.1145/1985793.1985795.
- [47] OpenAI, "Introducing GPT-5." 2025.
- [48] OpenAI, "Introducing GPT-4.1 in the API." Apr. 2025.
- [49] H. Liu *et al.*, "RefBERT: A Two-Stage Pre-trained Framework for Automatic Rename Refactoring." 2023.
- [50] U. Alon, S. Brody, O. Levy, and E. Yahav, "code2seq: Generating Sequences from Structured Representations of Code," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=H1gKY09tX>
- [51] W. A. M. C. Ouédraogo, K. Li, and others, "Human-Aligned Code Readability Assessment with Large Language Models." 2025.
- [52] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012. doi: 10.1007/978-3-642-29044-2.
- [53] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [54] I. Olkin and J. W. Pratt, "Unbiased Estimation of Certain Correlation Coefficients," *Annals of Mathematical Statistics*, vol. 29, no. 1, pp. 201–211, 1958, doi: 10.1214/aoms/1177706717.
- [55] R. A. Fisher, "Frequency Distribution of the Values of the Correlation Coefficient in Samples from an Indefinitely Large Population," *Biometrika*, vol. 10, no. 4, pp. 507–521, 1915, doi: 10.2307/2331838.
- [56] Vast.ai, "Vast.ai: GPU Cloud Computing Platform." 2024.
- [57] Q. Zhu and others, "DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence." 2024.
- [58] M. Abdin and others, "Phi-4 Technical Report." [Online]. Available: <https://arxiv.org/abs/2412.08905>
- [59] M. Abdin and others, "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone." [Online]. Available: <https://arxiv.org/abs/2404.14219>
- [60] Qwen Team, "Qwen3.5: A Family of Open-Source Multimodal Models." 2025.

APPENDIX A: PROMPT TEMPLATES

The pipeline uses three prompt templates. Prompt 1 is sent as the system instruction to establish the model's role and output constraints. Prompt 2 is the user-facing prompt providing the obfuscated test method and identifier list. Prompt 3 is the retry prompt, invoked when a response fails schema validation following the Reflexion framework [30].

APPENDIX A.1: SYSTEM PROMPT

```
You are a Java test code refactoring expert.
You will be given:
- A Java test method (wrapped in a dummy class), and
- A list of identifiers to rename, each tagged with its kind: (method) or (variable).

Your job is to propose meaningful names for ALL of the listed identifiers.

Naming conventions to follow:
- Methods (func_*): use camelCase starting with 'test',
  e.g.
  testShouldReturnNullWhenInputIsEmpty.
- Variables used in assertions (var_*): prefer 'expected' / 'actual' where applicable.
- Other variables (var_*): use concise camelCase nouns that describe the value,
  e.g. userCount, resultList.

Rules:
- You MUST include every identifier from the list as a key - no omissions.
- Use ONLY the listed identifiers as keys - no extras.
- You MUST ONLY respond with a valid JSON object mapping originalName -> newName.
- Do NOT output code, comments, markdown, or any text outside the JSON object.
```

Prompt 1. System Instruction Prompt

APPENDIX A.2: USER PROMPT

Here is the obfuscated Java test method wrapped in a dummy class:

```
```java
{test_case}
```
```

Here are ALL the identifiers that must be renamed:

```
{identifiers}
```

Rename ALL of the identifiers above. You MUST include every one of them as a key. Return a single JSON object mapping originalName -> newName.

Example (for a method + two variables):

```
{"func_1": "testShould...",
 "var_1": "actualResult",
 "var_2": "expectedSize"}
```

Rules:

- ALL listed identifiers must appear as keys in your response.
- Use ONLY the listed identifiers as keys - do NOT add or remove any.
- Do NOT output anything except the JSON object (no backticks, no text).

Prompt 2. User Prompt Template

APPENDIX A.3: RETRY PROMPT

Here is the original obfuscated Java test method wrapped in a dummy class:

```
```java
{test_case}
```
```

Here are ALL the identifiers that must be renamed:
{identifiers}

Your previous response was rejected for this reason:
{error_reason}

Your previous (rejected) response was:
{failed_response}

Please try again. Make sure your new response:

- Includes EVERY identifier from the list above as a key.
- Uses ONLY the listed identifiers as keys.
- Is a valid JSON object only - no backticks, no explanation.

Prompt 3. Retry Prompt Template

APPENDIX B: TRAINING CONFIGURATION

Adapters are injected into all seven projection matrices of each Transformer block (q_proj , k_proj , v_proj , o_proj , $gate_proj$, up_proj , $down_proj$), which has been shown to improve downstream performance over targeting attention projections alone [53]. The adapter parameterization follows LoRA [53]: for a weight matrix $W \in \mathbb{R}^{d \times k}$, the update is decomposed as $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$. Optimization uses paged AdamW in 8-bit precision with a cosine learning rate schedule and a 3% linear warmup.

APPENDIX C: GRADING TOOL CONFIGURATION

Listing 1 and Listing 2 show the complete configuration file used for CodeReader in this study. The configuration is split into two parts: the scoring and rules definition (Listing 1), and the model ensemble definition (Listing 2).

The scoring section defines a 0–100 scale with identifier naming as the primary focus. The four weighted criteria reflect their relative importance to the renaming task: identifier naming clarity carries the dominant weight (0.75) as it directly measures the quality of the renaming output; behavioral specificity of the test method name (0.15) captures whether the test scenario is communicated through the method name following the established JUnit convention [1]; and assertion quality (0.10) provides a secondary signal on the expressiveness of the test body [37]. Test independence is assigned a weight of

zero since the tool evaluates each test method in isolation, making cross-test dependencies unobservable and the criterion inapplicable in this setting.

The eight evaluation rules are derived from established research on identifier quality and test readability, and operationalise the scoring criteria as concrete, independently assessable instructions following the G-Eval recommendation [34], which shows that explicit fine-grained criteria produce more stable and calibrated scores than vague numeric scales.

Rules 1 and 2 — Clarity and penalization of auto-generated names. Butler et al. demonstrated empirically that poor identifier naming directly degrades a developer’s ability to understand and maintain code [11]. Lin et al. further showed that only 61.6% of test identifiers are rated acceptable compared to 92.5% in production code [12], confirming that naming quality in test code is a persistent problem. These findings motivate explicitly penalizing names that appear still auto-generated (e.g., `var0`, `obj1`), as these represent a complete failure of the renaming step.

Rule 3 — Cross-identifier consistency within a single test. A test method where naming quality is uneven across its own variables is harder to understand than one where all identifiers follow a coherent strategy, even if individual names score well in isolation. Scalabrino et al. identified intra-method naming consistency as a readability factor [36], and Lin et al. found that inconsistent naming within test code is a persistent quality problem [12]. This is distinct from file-level consistency: the rule penalizes tests where, for example, one variable is named `expectedUserAge` while a semantically related variable in the same method is named `result2`, regardless of how other test methods in the file are named.

Rule 4 — Alignment with actual test logic. A name that does not match what a variable actually holds is actively misleading and degrades comprehension more than a generic name would [11]. Scalabrino et al. include semantic alignment between identifiers and their role in the code as a key readability dimension [36].

Rule 5 — Test method naming and behavioral specificity. Daka et al. established that test method names following the pattern `methodName_stateUnderTest_expectedBehavior` significantly improve developer understanding of the tested behaviour, and that names like `test42` serve neither documentation nor comprehension purposes [1]. Winkler et al. confirmed that method naming is among the primary readability factors in test code [13].

Rule 6 — Assertion quality and magic numbers. Takebayashi et al. found that expressive assertions are an important but frequently overlooked readability factor in test code, and that hard-coded literals without named constants make the intent of assertions non-obvious [37].

Rule 7 — Test independence. Although test independence is weighted zero in our configuration because the tool evaluates each test method in isolation, the rule is retained in the rubric to maintain completeness and to allow future configurations that evaluate test files as a whole.

Rule 8 — Ignore class name. The enclosing class name is excluded from scoring because it falls outside the scope of the renaming task, which targets only local variable and

test method names. Penalizing a generic class name would introduce noise into the score that does not reflect the quality of the renaming output.

```
language: java

settings:
  timeout_seconds: 320
  health_timeout_seconds: 900
  max_concurrency: 1
  fail_on_no_active_models: true
  log_path: out/temp/readability_results.txt
  debug_jsonl_path: responses.jsonl

scoring:
  scale: 0-100
  focus: identifier_naming
  weights:
    identifier_naming: 0.75
    assertion_quality: 0.10
    test_independence: 0
    behavioral_specificity: 0.15

tags:
  - testing
  - unit tests
  - identifiers
  - function naming
  - variable naming
  - assertion clarity
  - test independence

rules:
  - Clarity of variable and method names assess whether identifier names clearly convey intent without needing to read the full test body. Penalise generic names like var1, obj, result0, func_1 or single letters unless used as counters.
  - Renamed vs. generic baseline explicitly penalise identifiers that appear to still be auto-generated (e.g. var0, obj1, int0, string1, list0). These represent failures of the renaming step and must receive the maximum penalty on the identifier dimension.
  - Identifier consistency across the test all identifiers within a single test method should follow a coherent naming strategy. If one variable is named expectedUserAge, a related variable should not be named result2. Penalise tests where naming quality is uneven across variables.
  - Alignment with actual test logic identifier names must match what the variable actually holds or what the method actually does. A variable named expectedResult holding an exception object is misleading and must be flagged.
  - Test method naming and behavioral specificity the test method name should follow the pattern methodName_stateUnderTest_expectedBehavior or a similar convention. The name and body together must make the tested behaviour and expected outcome obvious without reading the production code.
  - Assertion quality and magic numbers assertions must be expressive and clearly state what is being verified. Prefer assertEquals, assertNotNull, assertThrows over assertTrue(a == b). Penalise absent assertion messages when the failure reason would be non-obvious. Flag hardcoded numeric or string literals with no named constant or comment explaining their meaning
  - well-named expected value variables (e.g. int expectedAge = 42) are strongly preferred.
  - Test independence the test must be self-contained and not rely on shared mutable state or execution order. Penalise tests that depend on external state or side effects from other tests.
  - Ignore class name do not penalise or reward based on the outer class name.
```

Listing 1. Codereader Configuration File (codereader.yml) — Part 1

APPENDIX D.2: RANDOOP

```
models:
- name: Deepseek
  model: deepSeek-coder:6.7b
  runner: ollama
  runner_config:
    ollama_bin: ollama
  weight: 1.4
  enable: true

- name: Phi
  model: phi3:medium
  runner: ollama
  runner_config:
    ollama_bin: ollama
  weight: 0.75
  enable: true

- name: Qwen3.5
  model: qwen3.5:9b
  runner: ollama
  runner_config:
    ollama_bin: ollama
  weight: 1.25
  enable: true
```

Listing 2. Codereader Configuration File (codereader.yml) — Part 2

APPENDIX D: JAVA TEST EXAMPLES

In this section we show a couple of examples of how these test cases look like when auto-generated.

APPENDIX D.1: EVOSUITE

```
@Test(timeout = 4000)
public void test42() throws Throwable {
    Calculator calculator0 = new
    Calculator();

    double
    double0 = calculator0.modulo(1.0,
    1408.0511499512309);

    assertEquals(1,
    calculator0.getOperationCount());
    assertEquals(1.0, double0, 0.01);
}
```

Listing 3. Example of an EvoSuite-generated test before renaming

```
@Test
public void test011() throws Throwable {
    if (debug)
        System.out.format("%n%s%n",
        "RegressionTest0.test011");
    Calculator calculator0 = new
    Calculator();
    calculator0.reset();
    java.lang.String str3 =
    calculator0.classify((double) (byte)
    10);
    calculator0.memoryStore(0.0d);
    calculator0.memoryClear();
    double double9
    = calculator0.max((double) 10.0f,
    (-1.0d));
    java.lang.Class<?> wildcardClass10
    = calculator0.getClass();
    org.junit.Assert.assertEquals("'"
    + str3 + "' != '" + "medium" + "'",
    str3, "medium");
    org.junit.Assert.assertTrue("'" +
    double9 + "' != '" + 10.0d + "'", double9
    == 10.0d);
    org.junit.Assert.assertNotNull(
    wildcardClass10
    );
}
```

Listing 4. Example of a Randoop-generated test before renaming

APPENDIX E: DATASET

The dataset is constructed by obfuscating human-written test code to simulate the generic identifier naming produced by automated test generation tools. While this approximation does not fully capture the structural characteristics of tool-generated tests, which may vary depending on the generator used, it serves the primary purpose of this study: exposing the model to the pattern of uninformative variable and method names that automated tools produce, such as `var_0` and `func_1`, without access to any semantic context.

APPENDIX E.1: OBFUSCATED CODE

```
public class TestClass10000 {
    @Test public void func_1() {
        Map<Integer, String> var_1 =
            new HashMap<Integer, String>();
        var_1.put(1, "one");
        var_1.put(2, "two");
        var_1.put(3, "three");
        var_1.put(4, "four");
        Assert.assertEquals(
            "{ 1 -> one , 2 -> two ,
              3 -> three , 4 -> four }",
            MapUtils.toString(var_1));
    }
}
```

Listing 5. An example of obfuscated test case

APPENDIX E.2: JSONL FILE

```
{
    "prompt": "public class
TestClass10000 {\n@Test public void
func_1() { Map<Integer, String>
var_1 = new HashMap<Integer,
String>(); var_1.put(1, \"one\");
var_1.put(2, \"two\"); var_1.put(3,
\"three\"); var_1.put(4, \"four\");
Assert.assertEquals(\"{ 1 -> one , 2 -
> two , 3 -> three , 4 -> four }\",
MapUtils.toString(var_1)); }\n",
    "response": "public class
TestClass10000 {\n@Test public void
shouldPrettyPrintMap() { Map<Integer,
String> map = new HashMap<Integer,
String>(); map.put(1, \"one\");
map.put(2, \"two\"); map.put(3,
\"three\"); map.put(4, \"four\");
Assert.assertEquals(\"{ 1 -> one , 2 -
> two , 3 -> three , 4 -> four }\",
MapUtils.toString(map)); }\n"
}
```

Listing 6. An example Jsonl dataset sample

APPENDIX F: SAMPLE SIZE JUSTIFICATION

This appendix provides the statistical justification for the choice of 100 evaluation samples per checkpoint, as referenced in the Dataset section.

APPENDIX F.1: METHOD

Effect sizes are computed using the Wilcoxon signed-rank test on paired observations, following the non-parametric approach recommended for software engineering experiments with non-normal distributions [46]. The effect-size statistic is $r = \sqrt{\frac{Z}{N}}$, interpreted as small ($|r| \geq 0.10$), medium ($|r| \geq 0.30$), or large ($|r| \geq 0.50$) following Cohen [45]. To correct

for small-sample inflation, r is bias-adjusted using the Olkin–Pratt estimator [54]. The 95% confidence interval on r is obtained via the Fisher Z -transformation [55]. The recommended sample size n is derived from the conservative lower bound of this confidence interval, converted to Cohen’s d and passed through the standard formula for a two-sided test at $\alpha = 0.05$, power = 0.80, with a 25% non-normality correction applied to account for the Wilcoxon efficiency ratio relative to a t -test.

The pilot run used to estimate effect sizes consisted of 30 randomly sampled test methods evaluated across all six Qwen2.5-Coder checkpoints. Effect sizes from this pilot are conservative inputs to the power analysis: using the lower bound of the 95% confidence interval on r ensures that the recommended n accounts for uncertainty in the pilot estimate itself.

APPENDIX F.2: PRIMARY COMPARISONS

Table IV reports the recommended n for all comparisons that directly answer the research questions. All effects are large and highly significant ($p < 0.001$); the recommended n never exceeds 10 across any checkpoint.

TABLE IV
RECOMMENDED n FOR PRIMARY COMPARISONS ACROSS FINE-TUNING CHECKPOINTS. ALL EFFECTS ARE LARGE ($|r| \geq 0.50$, $p < 0.001$).

| Comparison | 0% | 15% | 25% | 50% | 75% | 100% |
|--------------------------|----|-----|-----|-----|-----|------|
| F1 vs zero baseline | 2 | 2 | 2 | 2 | 2 | 2 |
| LLM avg: obf vs renamed | 2 | 4 | 4 | 3 | 7 | 3 |
| LLM wavg: obf vs renamed | 3 | 5 | 5 | 4 | 10 | 3 |
| LLM avg: obf vs oracle | 2 | 3 | 2 | 3 | 3 | 2 |
| LLM wavg: obf vs oracle | 3 | 3 | 3 | 4 | 4 | 3 |

APPENDIX F.3: QUALITY GAP COMPARISON

Table V reports the renamed vs. oracle comparison, the most demanding one, measuring how close pipeline output comes to human-written quality.

TABLE V
RECOMMENDED n AND SIGNIFICANCE FOR THE RENAMED VS. ORACLE COMPARISON. † MARGINALLY SIGNIFICANT; †† NOT SIGNIFICANT. LARGE VALUES REFLECT SMALL, FRAGILE EFFECTS RATHER THAN A SYSTEMATIC GAP.

| Comparison | 0% | 15% | 25% | 50% | 75% | 100% |
|-----------------------------|-------|-----|-----|-------|-----|------|
| LLM avg: renamed vs oracle | 2085† | 236 | 77 | 345 | 56 | 77 |
| LLM wavg: renamed vs oracle | 400†† | 561 | 90 | 2950† | 57 | 236 |

Three observations justify accepting 100 samples despite these large values for some conditions. First, the effects driving the large recommended n are small and statistically fragile: at 0%, the weighted-average variant ($r = -0.13$, $p = 0.20$) is not significant at all, and at 50% the same variant ($r = -0.22$, $p = 0.027$) is similarly weak. Second, this instability is itself a

finding, as the base and partially fine-tuned models produce variable-quality output that is sometimes near-oracle and sometimes substantially below it, a characteristic that additional samples would not resolve. Third, the compute cost makes scaling infeasible, as shown in Table VI.

TABLE VI

WALL-CLOCK RUNTIME PER 100-SAMPLE RUN AND ESTIMATED TIME TO REACH THE MAXIMUM RECOMMENDED n OF 2 085.

| Models | Runtime (100 samples) | Est. for 2 085 samples |
|----------------|-----------------------|------------------------|
| Qwen zero-shot | 6.4 h | ~133 h |
| Qwen 15% | 5.3 h | — |
| Qwen 25% | 4.5 h | — |
| Qwen 50% | 4.8 h | — |
| Qwen 75% | 5.7 h | — |
| Qwen 100% | 3.3 h | — |
| GPT-4.1 mini | 5.5 | — |
| GPT-5 mini | 6.22 h | — |

Conclusions about the absolute quality gap between re-named and oracle output at underpowered checkpoints should therefore be treated as indicative rather than definitive.

APPENDIX G: BENCHMARK RESULTS

APPENDIX G.1: QWEN2.5-CODER

Table VII reports the complete set of quantitative metrics for all Qwen2.5-Coder checkpoints. CER and exact match scores complement the F1 and LLM readability scores reported in Table III. The oracle columns report the LLM readability score of the original human-written test methods on the same subset, serving as an upper bound for the LLM score.

TABLE VII

FULL BENCHMARK METRICS PER CHECKPOINT. EM-ORD = ORDERED EXACT MATCH, EM-UNORD = UNORDERED EXACT MATCH, CER = CHARACTER ERROR RATE.

| Ckpt | P | R | F1 | EM-ord | EM-unord | CER |
|-------|-------|-------|-------|--------|----------|-------|
| 0% | 0.714 | 0.817 | 0.760 | 0.753 | 0.730 | 32.03 |
| 68.13 | 15% | 0.848 | 0.825 | 0.834 | 0.785 | 0.792 |
| 17.53 | 67.89 | 25% | 0.853 | 0.831 | 0.839 | 0.793 |
| 0.804 | 16.94 | 68.21 | 50% | 0.849 | 0.828 | 0.836 |
| 0.785 | 0.801 | 17.54 | 68.13 | 75% | 0.853 | 0.824 |
| 0.836 | 0.785 | 0.798 | 17.18 | 67.31 | 100% | 0.849 |
| 0.826 | 0.835 | 0.785 | 0.793 | 17.27 | 68.43 | |

When evaluating the Oracle LLM readability score, this averaged out over 6 runs to 68.02.

APPENDIX H: COMPUTE ENVIRONMENT

All fine-tuning and inference experiments for Qwen2.5-Coder were conducted on cloud computing infrastructure, as full fine-tuning and checkpoint evaluation at the 7B parameter scale exceeded the capacity of available local hardware. The CodeReader judge ensemble was served together on the cloud via Ollama during evaluation.

TABLE VIII

COMPUTE ENVIRONMENT DETAILS USED ON VASTAI CLOUD SERVICES.

| Component | Details |
|------------------|---------------|
| Cloud provider | VastAI [56] |
| GPU type | NVIDIA 5090 |
| GPU memory | 32 GB |
| Fine-tuning runs | 6 checkpoints |

All OpenAI evaluations had to be done on the local environment, because OpenAI blocks API calls from cloud providers.

TABLE IX

COMPUTE ENVIRONMENT DETAILS USED ON LOCAL ENVIRONMENT.

| Component | Details |
|------------|-------------|
| GPU type | NVIDIA 3090 |
| GPU memory | 12 GB |

APPENDIX I: MODEL USAGE STATISTICS

Table X reports token usage and latency statistics for all Qwen2.5-Coder checkpoints, GPT-5 mini and GPT 4.1 mini.

A notable difference emerges between the two model families. Qwen2.5-Coder produces compact JSON completions averaging 43–48 tokens per file, with fine-tuned checkpoints generating slightly shorter responses than the untuned baseline (43–46 vs. 47.8 tokens), consistent with the model learning more concise naming patterns. Similar to Qwen2.5-Coder, GPT-4.1 mini produces short sets of tokens per file, on average 37 tokens. GPT-5 mini, by contrast, produces substantially more verbose completions averaging 562.6 tokens per file, roughly 12 times more than any Qwen checkpoint. Inspection of the raw responses reveals that GPT-5 mini does not confine itself to the requested JSON object but frequently includes reasoning, explanations, and commentary alongside the mapping, despite the prompt explicitly instructing JSON-only output.

GPT-5 mini averages 9,076 ms per file, which is comparable to Qwen’s 3,000–10,000 ms range, though Qwen exhibits greater variance due to cloud infrastructure fluctuations. GPT-4.1 mini stands out with a markedly lower average latency of 1,333 ms, roughly seven times faster than GPT-5 mini. This gap is likely attributable to GPT-5 mini’s internal reasoning mechanism, which incurs additional compute overhead before producing a response.

All models achieved a 100% success rate across 100 files.

TABLE X

TOKEN USAGE AND LATENCY STATISTICS PER CHECKPOINT. PROMPT TOKENS ARE NEAR-IDENTICAL ACROSS ALL MODELS AS THE SAME INPUT FILES AND PROMPT TEMPLATE WERE USED.

| Model | Prompt tokens | Completion tokens | Total tokens | Avg latency (ms) | Success |
|--------------|---------------|-------------------|--------------|------------------|---------|
| Qwen (0%) | 53,053 | 4,779 | 57,832 | 2,940 | 100/100 |
| Qwen (15%) | 53,053 | 4,333 | 57,386 | 7,582 | 100/100 |
| Qwen (25%) | 53,053 | 4,589 | 57,642 | 8,274 | 100/100 |
| Qwen (50%) | 53,053 | 4,387 | 57,440 | 7,044 | 100/100 |
| Qwen (75%) | 53,053 | 4,488 | 57,541 | 9,943 | 100/100 |
| Qwen (100%) | 53,053 | 4,542 | 57,595 | 4,779 | 100/100 |
| GPT-4.1 mini | 53,343 | 3,708 | 57,051 | 1,333 | 100/100 |
| GPT-5 mini | 53,243 | 56,263 | 109,506 | 9,076 | 100/100 |

APPENDIX J: MODEL SELECTION: QWEN2.5-CODER

At the time of model selection in November 2025, the Qwen2.5-Coder family was the leading open-source code model series across its size range. The 7B variant was selected based on the hardware available at that stage of the project: a local setup on which larger variants were not practically deployable. The 14B model requires approximately 9 GB of VRAM for inference and the 32B model approximately 20 GB, both exceeding local capacity, while the 7B model (approx. 4.7 GB for inference) remained feasible. Fine-tuning was subsequently performed on a cloud environment, where even the 7B model required approximately 20 GB of VRAM under QLoRA, confirming that the larger variants would not have been viable under the original local constraints in any case. Full details of the compute environment used for fine-tuning are provided in Appendix H.

TABLE XI

QWEN2.5-CODER BENCHMARK SCORES ACROSS MODEL SIZES AT TIME OF RELEASE (NOVEMBER 2024). BOLD VALUES INDICATE THE BEST SCORE PER BENCHMARK ROW.

| Benchmark | 32B | 14B | 7B | 3B | 1.5B | 0.5B |
|---------------|-------------|-------------|------|------|------|------|
| HumanEval | 92.7 | 89.6 | 88.4 | 84.1 | 70.7 | 61.6 |
| MBPP | 90.2 | 86.2 | 83.5 | 73.6 | 69.2 | 52.4 |
| EvalPlus | 86.3 | 84.0 | 81.9 | 75.2 | 66.5 | 53.8 |
| MultiPL-E | 79.4 | 79.6 | 76.5 | 72.1 | 56.7 | 49.6 |
| McEval | 65.9 | 61.9 | 60.3 | 42.6 | 33.8 | 18.7 |
| LiveCodeBench | 31.4 | 23.4 | 18.2 | 10.8 | 6.1 | 2.0 |
| CRUXEval-O | 83.4 | 79.5 | 65.9 | 56.0 | 37.5 | 27.8 |
| BigCodeBench | 38.3 | 36.3 | 29.8 | 29.4 | 37.6 | 34.5 |
| Spider | 85.1 | 84.8 | 83.1 | 76.8 | 68.8 | 55.7 |
| BIRD-SQL | 58.4 | 56.9 | 52.5 | 38.7 | 28.4 | 15.1 |
| CodeArena | 68.9 | 50.0 | 43.1 | 28.3 | 13.3 | 5.5 |

APPENDIX K: MODEL SELECTION: READABILITY TOOL

DeepSeek-Coder 6.7B was preferred over its successors, because its successors consumed too much vram to be ran on the local hardware together with the other models. A quick estimate was that its direct successor (DeepSeek-Coder V2), whose smallest available variant has 16B parameters and requires approximately 10.4 GB of VRAM at Q4, leaving insufficient headroom for the KV cache on the available hardware and causing out-of-memory errors in practice [57].

Phi-3-medium was preferred over Phi-4 on hardware and task-fit grounds. Phi-4 at Q4 quantization requires approximately 10.5 GB of VRAM, leaving only approximately 1.5 GB for the KV cache, which caused out-of-memory errors during evaluation [58]. Phi-3-medium’s strong instruction-following capability, evidenced by an MT-bench score of 8.9 out of 10 [59], makes it well suited to consistently applying a structured grading rubric across diverse test methods.

Qwen3.5-9B was selected as the third ensemble member over earlier Qwen variants on the basis of its improved general reasoning capabilities and compact hardware footprint of approximately 6 GB at Q4, leaving ample headroom within the 12 GB budget. Its inclusion provides complementary coverage of natural language criteria such as behavioral specificity, which are less central to a code-specialized model [60].